

Pruebas de diferentes Prompt

Se verá cómo cambiando el prompt en diferentes situaciones a gemini puede cambiar estos resultados.

Diferentes escenarios y prompts

- **Escenario 1:** Este escenario es demostrar que tan diferente es el resultado variando el contexto.

Primer Prompt: Sin contexto y solo dando la pregunta

Pregunta: "¿Qué es un hook_menu en Drupal 7?"

Resultado:

En Drupal 7, el `hook_menu()` es uno de los hooks más importantes y fundamentales. Su función principal es **registrar rutas de URL (direcciones web)** y **definir qué debe pasar cuando un usuario las visita**.

En términos sencillos, actúa como el sistema de enrutamiento del sitio, pero con superpoderes, ya que hace mucho más que mapear una URL.

¿Para qué sirve? (Funciones principales)

Cuando defines una ruta dentro de `hook_menu()`, puedes especificar:

- **Mapeo de URL a funciones:** Le dice a Drupal: "Cuando alguien visite `misitio.com/mi-pagina`, ejecuta la función `mi_modulo_pagina_callback()`".
- **Creación de menús:** Puede colocar automáticamente esa URL como un enlace en un menú (como el Menú de Navegación, el Menú Principal o el de Administración).
- **Pestañas locales (Local tasks):** Permite crear esas pestañas típicas de Drupal que ves al editar un nodo (Ver, Editar, Revisar).
- **Control de acceso:** Define qué permisos necesita el usuario para poder ver esa página (por ejemplo, solo administradores).

El resultado es más simple, es una definición puramente técnica y arreglos asociativos sin un fin práctico aparente. Además, de que da el código de ejemplo de lo que se preguntó.

Segundo Prompt: Con contexto sobre el rol que va a cumplir y como responder

Pregunta: "Actúa como un Profesor de Ingeniería de Software. Explícame de forma didáctica y con una analogía de la vida real qué es y para qué sirve un hook_menu en Drupal 7."

Resultado:

¡Buenas! Qué gran tema traes a la clase de hoy. Sentate y acomodate. Como tu profesor de Ingeniería de Software, te aseguro que entender el `hook_menu` en Drupal 7 es el "Eureka" que necesitás para dominar la arquitectura de este CMS.

Para entenderlo sin complicarnos con código desde el inicio, vamos a sacarlo del entorno de servidores y llevarlo a la vida real con una analogía muy cotidiana.

La Analogía: El Recepcionista de un Edificio de Oficinas

Imaginá que entrás a un edificio corporativo enorme. Vos vas con una intención clara: querés hablar con el contador para ver el estado de tus finanzas.

En la entrada, hay un **Recepcionista** con una libreta enorme. El proceso funciona así:

- 1 Vos llegás y decís: *"Quiero ir a la oficina de Finanzas"* (Eso en la web sería escribir una URL en el navegador: `misitio.com/finanzas`).
- 2 El recepcionista busca en su libreta la palabra **"Finanzas"** (La ruta).
- 3 Al encontrarla, revisa sus anotaciones:
 - **¿Quién tiene permitido pasar?** Revisa si tenés el carnet adecuado (Permisos de acceso / *Access arguments*).
 - **¿A dónde te tiene que mandar?** Si todo está en orden, te dice: *"Suba al piso 3, oficina 302 y hable con don Carlos"* (Esta es la función callback que procesa la lógica).
- 4 Vos vas a la oficina 302, don Carlos hace su trabajo y te entrega tu reporte (El contenido de la página).

El modelo adopta un tono educativo. Utiliza la analogía de un "Recepcionista de una oficina" para explicar que asocia una URL con una acción, facilitando la comprensión inmediata del estudiante.

Comparación de los dos resultados: Dependiendo de lo que se le ponga en el prompt se denota, como responde el modelo y además dependiendo de lo que se ponga en el prompt este te puede agregar nuevas cosas al resultado.

- **Escenario 2:** Este escenario es evaluar la capacidad de la IA para estructurar código útil bajo reglas estrictas versus peticiones abiertas.

Primer prompt: Es un prompt seguro que no especifica nada de que se quiere hacer además de las herramientas que se usan.

Pregunta: "Dame un código para limpiar malas palabras en Drupal."

Resultado:

En Drupal, la forma más limpia y profesional de manejar esto no es escribiendo código desde cero, sino aprovechando el sistema de Hooks o creando un **Constraint (validador)** si estás en una versión moderna (Drupal 8, 9, 10 u 11).

Como no especificaste la versión, te comparto dos soluciones: la forma nativa usando un módulo existente (lo ideal para producción) y la forma en código mediante un módulo personalizado para Drupal 9/10/11.

```
PHP

<?php

use Drupal\Core\Form\FormStateInterface;

/**
 * Implements hook_form_alter().
 */
function mis_filtros_form_alter(&$form, FormStateInterface $form_state, $form_id) {
  // Puedes cambiar esto por el ID del formulario que quieras controlar
  // Ejemplo: 'comment_comment_form' para comentarios o 'node_article_form'
  if ($form_id == 'comment_comment_form' || $form_id == 'node_page_form') {
    // Añadimos nuestra función de validación al principio del array
    array_unshift($form['#validate'], 'mis_filtros_validar_malas_palabras');
  }
}
```

El modelo devuelve un script genérico de PHP o un ejemplo para versiones modernas de Drupal usando programación orientada a objetos, el cual no es compatible con el proyecto actual que es Drupal 7.

Segundo prompt: Es un prompt seguro que no especifica nada de que se quiere hacer además de las herramientas que se usan.

Pregunta: "Escribe una función de PHP estrictamente para Drupal 7 utilizando el hook_comment_presave. El código debe evaluar en minúsculas si el cuerpo del comentario contiene la palabra 'porqueria' y reemplazar todo el texto por un aviso institucional. Agrega comentarios explicativos en cada línea y no uses funciones externas de PHP que requieran librerías adicionales, solo usando un módulo custom."

Resultado:

Aquí tienes la implementación del hook estrictamente para Drupal 7.

En Drupal 7, el cuerpo de los comentarios utiliza la API de campos (*Field API*), por lo que la estructura de los datos es un arreglo multidimensional que incluye el idioma y el valor del campo (usualmente `comment_body`).

Copia este código dentro del archivo `.module` de tu módulo personalizado (reemplaza `mi_modulo_custom` por el nombre real de tu módulo):

```
/**
 * Implementación de hook_comment_presave().
 * * Este hook se ejecuta justo antes de que el comentario sea guardado
 * o actualizado en la base de datos.
 */
function mi_modulo_custom_comment_presave($comment) {
  // 1. Verificamos si existe el campo del cuerpo del comentario y si no está vacío.
  if (!empty($comment->comment_body)) {

    // 2. Obtenemos el idioma actual del campo (usualmente 'und' para idioma indeterminado)
    $idioma = field_language('comment', $comment, 'comment_body');

    // 3. Extraemos el texto original del cuerpo del comentario usando el idioma detectado
    $texto_original = $comment->comment_body[$idioma][0]['value'];

    // 4. Convertimos el texto a minúsculas usando una función segura para UTF-8 (multibyte)
    // lo que evita problemas con acentos o caracteres especiales en español.
    $texto_en_minusculas = mb_strtolower($texto_original, 'UTF-8');

    // 5. Buscamos si la palabra prohibida 'porqueria' existe dentro del texto en minúsculas
    // Usamos strpos, que devuelve la posición del texto o FALSE si no lo encuentra.
    if (strpos($texto_en_minusculas, 'porqueria') !== FALSE) {
```

Detalles importantes a tener en cuenta:

- **Uso de `mb_strtolower`** : Al especificar `'UTF-8'` , nos aseguramos de que si un usuario escribe "PORQUERÍA" (con tilde), la conversión no rompa los caracteres especiales, aunque la búsqueda exacta que pediste es para el término sin tilde (`'porqueria'`).
- **Estructura de Fields:** En Drupal 7, acceder directamente a `$comment->comment_body['und'][0]['value']` es una práctica común, pero usar `field_language()` es la forma correcta y nativa de Drupal para evitar errores si el sitio se vuelve multi-idioma en el futuro.

El modelo entrega exactamente el bloque de código limpio, compatible con la API de Drupal 7, acotado a la lógica de negocio requerida y listo para ser pegado en el archivo `".module"`.